

Original papers

Digital twin-driven system for efficient tomato harvesting in greenhouses

Yining Lang^{a,b}, Yanqi Zhang^{a,b}, Tan Sun^a, Xiujuan Chai^a, Ning Zhang^{a,b}^a Agricultural Information Institute, Chinese Academy of Agricultural Sciences, Beijing 100081, China^b Key Laboratory of Agricultural Big Data, Ministry of Agriculture and Rural Affairs, Beijing 100081, China

ARTICLE INFO

Keywords:

Tomato
Harvest
Digital-twin
Greenhouse
Reinforcement-learning
Decision

ABSTRACT

Efficient and low-damage harvesting remains a major challenge in modern greenhouse tomato production, particularly in dense planting environments. To address limitations such as restricted camera views, occluded fruits, and complex fruiting patterns, our study presents a digital twin-driven system for intelligent tomato harvesting. Using a slidable depth camera mounted on the robot, we reconstruct a high-fidelity 3D digital twin of the greenhouse that accurately captures the spatial distribution and growth states of tomatoes. Based on this virtual environment, a learning-based framework is developed to optimize harvesting strategies, including robot positioning, arm trajectory planning, fruit selection priority, and adaptive operation modes. The proposed system integrates both a complete algorithmic workflow and a practical hardware platform. Experimental results show that our method significantly improves harvesting performance, reducing the average harvesting time by 34.95% (to 7.4 s per fruit), arm movement distance by 20.93%, and collision occurrences by 45.16%. While tailored for tomato harvesting, this framework demonstrates strong potential for generalization to other greenhouse crops in precision agriculture.

1. Introduction

With the rapid development of artificial intelligence and hardware, harvesting robots are becoming increasingly important in digital agriculture. Tomatoes, a key crop in protected cultivation, are well-suited for standardized greenhouse production. Their dense distribution, varied maturity, and frequent occlusions present major challenges for 3D perception, path planning, and control (Li et al., 2024b; Zheng et al., 2024; Dong et al., 2025; Liu et al., 2024b; Su et al., 2023). These complexities make tomato harvesting a long-standing research focus and an ideal benchmark for developing intelligent systems with strong generalization potential. Early research primarily relied on traditional image processing methods, such as K-means clustering (Yin et al., 2009), combined with end-effector design (Sengaphone et al., 2024; Luo et al., 2023), to achieve tomato detection and harvesting. With the advancement of deep learning, detection accuracy has significantly improved (Zheng et al., 2024; Miao et al., 2023; Yonghong et al., 2024; Firdaus et al., 2024), and pose estimation techniques have been increasingly applied to cherry tomato harvesting, further boosting success rates (Li et al., 2024a; Rong et al., 2022; Kim et al., 2022; Dong et al., 2025). To further enhance efficiency, dual-arm harvesting robots (Li et al., 2024c) and systems that mimic human-like picking behaviors (Liu et al., 2024a) have also emerged in recent years. At the same time, the researchers (Gong et al., 2022) found that tomatoes

are easily obscured by leaves, which affects recognition accuracy and picking efficiency. Dai et al. (2024) propose a multi-view method to achieve more accurate recognition. Sun et al. (2023) design an object localization method in occluded agricultural environments through deep learning and active sensing.

Although recent works (Li et al., 2024a; Dai et al., 2024) have tried to improve the accuracy of tomato recognition as much as possible by localizing the key points and combining with the end-effector for better picking, there are still several major efficiency bottlenecks that have not been broken through. On the one hand, tomato picking robots need to recognize the position and ripeness of tomatoes before making a picking decision. However, due to the close spacing of the planting rows, in most greenhouses the camera view on the robot, in the case of the most widely used Intel D435 model, makes it difficult to present all the tomatoes in front of the robot in one complete view. This makes picking path planning within the localized viewpoint less useful, as there are often only a small number of tomatoes within the viewpoint. In mixed ripe/unripe areas, the robot may stop to pick only 1–3 tomatoes at a time. Also, after picking, the robot arm or chassis needs to be moved to continue “carpet” scanning for new clusters, thus reducing picking efficiency. On the other hand, when picking tomatoes, it is easy to encounter ripe tomatoes that are inside the seedling. This

* Corresponding author.

** Corresponding author at: Agricultural Information Institute, Chinese Academy of Agricultural Sciences, Beijing 100081, China.

E-mail addresses: chaixiujuan@caas.cn (X. Chai), zhangning@caas.cn (N. Zhang).

type of tomato is difficult to reach with a robotic arm, and there is a risk of damaging the seedlings, making it more suitable for manual supplemental picking. But current robots tend to waste a lot of time on these tomatoes until it is determined that they cannot be picked. In addition, tomatoes are characterized by multiple batches of fruits in a short period of time after reaching the fruiting stage, which makes it common to see ripe, half-ripe, and unripe tomatoes mixed together. In a large greenhouse, there is a lack of global consideration of how to monitor differences in ripeness between rows to prioritize harvesting and the optimal time to start harvesting. This can result in some fruit being overripe and falling off and rotting, or being harvested too early with too little ripeness for too low an input–output ratio.

The main reason why existing works cannot overcome these three efficiency challenges is that the system lacks a global view to aggregate the location and ripeness information of tomatoes and seedlings, so existing picking decisions are made only for a local area, not for the whole. However, a set of locally optimal solutions, measured in terms of efficiency, may not be globally optimal. When we tried to think globally about picking decisions, the 3D reconstruction and path planning algorithms came into view. Actually, there are some previous works that try to improve the efficiency problem of tomato picking by using 3D perception (Rapado-Rincon et al., 2024), reconstruction (Gong et al., 2022) or path planning (Qin et al., 2021). Nevertheless, these 3D-related works primarily focus on enhancing the algorithm's local detection capabilities by utilizing 3D technology for multi-dimensional perception or reconstruction of occluded areas, rather than simulating the entire greenhouse through 3D reconstruction. For path planning, it mainly focuses on local decisions within the camera's field of view. Li et al. (2024d) propose a stop-move motion planning method for tomato harvesting robots, using a projection map of each row to determine the optimal stopping position for picking the most fruits. While effective under ideal conditions, this approach may not work well in more complex environments, such as overlapping tomatoes, as it relies solely on 2D projection, which cannot accurately reflect actual distances or avoid 3D obstacles. This limits the effectiveness of the picking plan.

In this context, we argue that relying solely on 2D projections or local 3D simulations is insufficient to support efficient decision-making for tomato harvesting in complex greenhouse environments. It is imperative to develop a “digital twin greenhouse” system that incorporates complete spatial semantics and supports dynamic perception and interaction. Unlike traditional 3D modeling, a digital twin not only enables high-precision reconstruction of tomato plant positions and morphology, but also integrates real-time environmental sensing through sensors, dynamic simulation of fruit ripening, and continuous data updates and strategy feedback throughout multiple harvesting cycles. This provides a precise virtual environment for decision-making algorithms such as reinforcement learning. To this end, we leverage the digital twin greenhouse as the core platform — not only for 3D reconstruction and environmental modeling, but also as the training and application environment for decision-making algorithms — thus forming a closed loop between perception, simulation, and decision-making. This integrated approach enables end-to-end optimization from digital modeling to intelligent harvesting, ultimately improving picking efficiency while reducing damage to fruits and plants.

Motivated by these needs, our research aims to construct a high-fidelity digital twin greenhouse using multi-view scanning and 3D modeling technologies, and to train reinforcement learning models within this twin environment to make integrated harvesting decisions, such as determining harvesting timing, row-level prioritization under a global view, robot stop points, path planning, and manual re-harvesting. These trained models will then be deployed on actual harvesting robot systems, with their performance evaluated through both simulation and real-world experiments, focusing on aspects like harvesting efficiency, motion path length, and collision rate. This research is significant as it bridges the gap between local-view tomato harvesting strategies and global optimization by introducing a digital twin greenhouse system,

which integrates perception, simulation, and decision-making. It offers a new paradigm for large-scale deployment of intelligent agricultural systems driven by digital twin technologies.

Our main contributions are as follows: (1) Using dynamic crop scanning, recognition, and 3D reconstruction technologies, we present the first complete digital twin greenhouse system for tomato harvesting decision-making. (2) We introduce a reinforcement learning model within the digital twin greenhouse to jointly optimize robot positioning, arm trajectory planning, fruit selection priority, and adaptive operation modes, overcoming the limitations of prior local-view-based strategies. (3) We validate the system through both simulation and real-world deployment, demonstrating significant improvements in harvesting efficiency, reduced path length and plant collision rate, and better alignment between fruit maturity and harvesting timing, highlighting its broad potential for large-scale intelligent transformation in precision agriculture.

2. Materials and methods

As shown in Fig. 1, the system first scanned and recognized tomato data in the greenhouse using a slidable depth camera attached to the robot. It collected information such as ripeness, size, and the location of both tomatoes and seedlings. This information was pre-processed and reconstructed in the Unity engine to create the digital twin, with the tomatoes' growth state (from unripe to ripe size, including the gradual change in fruit color) simulated in real time based on historical greenhouse data. Using data from the virtual greenhouse, we trained a reinforcement learning model to guide the robot's decision-making, including parking position, picking order, whether to pick or not, and timing control.

2.1. Hardware devices

The tomato picking robot used in this work consists of four main modules: a robotic arm, a control computer, a scanning slider, and a chassis, as shown in Fig. 2(a). The robotic arm, model ELITE ROBOTS EC66, is equipped with a picking end-effector and an Intel RealSense D435 depth camera at the end. The robot hosts a control unit modeled by the NVIDIA Jetson AGX Orin, which has 275 TOPS, a 12-core ArmCortex-A78AE CPU, and 64 GB of video memory. The scanning slide is 1.2 m long and carries another depth camera of the same model, capable of automatically scanning back and forth across the slide at 10 cm/s and visualizing the scan data on the screen, as shown in Fig. 2(b). The chassis used in the experiment is tracked and equipped with two storage boxes. Its driving method and the number of storage bins can be dynamically adjusted based on the actual application site. The chassis can also be easily switched between different types, such as tracked or wheeled.

2.2. Data collection

The tomato variety used in this study is normal size tomato, rather than cherry-tomato, and the greenhouse is 60 m long and 30 m wide, with plants spaced approximately 2 m apart, including 0.8 m in the planting area and 1.2 m in the aisle, for a total of 30 rows.

The collected data consists of both RGB and depth information. The RGB data is utilized to recognize key attributes such as tomato ripeness, leaf densities, and the locations of tomatoes, leaf clusters, and tomato stems in the planar projection. Meanwhile, the depth images provide the necessary information to determine the spatial depth of tomatoes, leaf clusters, and tomato stems. To assess tomato ripeness and extract segmentation results, we apply the Mask R-CNN (He et al., 2017) algorithm on the RGB images. By combining the segmentation results with the corresponding depth data, we can accurately determine the 3D spatial locations of the tomatoes within the environment.

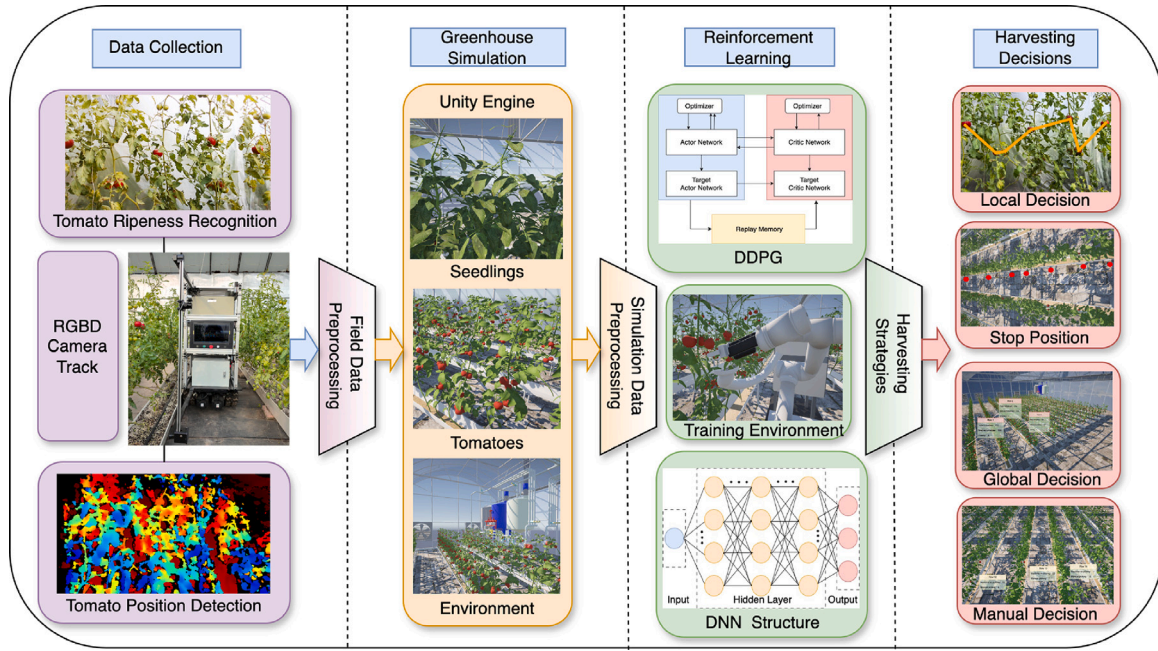


Fig. 1. Framework of the Digital Twin-Driven System, which consists of four parts: data collection, greenhouse simulation, reinforcement learning and harvest decision. In the data collection part, information such as the location and ripeness of the tomato is collected. After pre-processing, the data is reconstructed in the Unity engine to simulate the environment, seedlings and tomatoes. Based on the size, location and ripeness of the tomatoes, the reinforcement learning model is trained. The trained model network provides a complete harvesting decision including local path, stop position, global decision and automatic/manual picking decision.

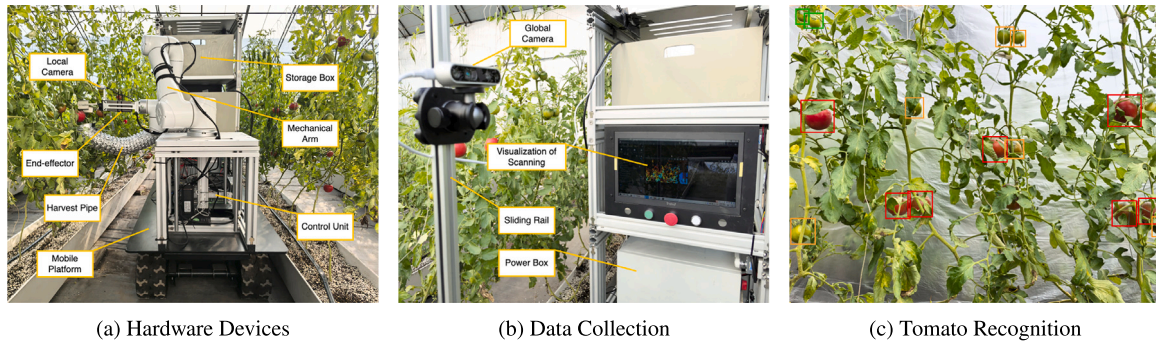


Fig. 2. Illustration of the tomato harvesting robot and its data collection system. (a) The robot's hardware setup, including a robotic arm with a controllable sliding harvesting pipe. (b) The data acquisition system utilizing a depth camera on a sliding rail for capturing RGB-D data. (c) The tomato recognition process, showing position detection and ripeness identification, the red box, yellow box, and green box represent ripe, semi ripe, and unripe, respectively.

Meanwhile, for the robotic arm, we utilize a YOLO-based (Ultralytics, 2024) architecture, commonly adopted in the field, specifically for fine-tuning the arm's posture during harvesting operations. This setup enables rapid real-time detection of tomato locations, assisting in precise adjustments rather than overall decision-making or path optimization.

Both the Mask R-CNN (He et al., 2017) model and the YOLO v8 (Ultralytics, 2024) model we used are trained based on pre-trained models under the tomato dataset. The dataset consists of 2000 tomato images captured from greenhouse shoots at all angles, including unobstructed and a small number of obstructed cases. We manually labeled the bounding box and segmentation regions, and the labeling rules were identical to those of the coco dataset (Lin et al., 2025). 80% of the dataset is used for training and 20% for validation.

It is worth noting that the data acquired by the slider must be aligned in the same coordinate system in order to acquire the complete plant (as shown in Fig. 3 left) data based on the sequence of acquired data frames. The coordinate system alignment steps are as follows: (1) The slider takes an image of the tomato plant in the low, middle, and high regions and collects the depth information; (2) The SIFT (Lowe,

1999) feature point matching algorithm is used to match the feature points for the two image sets, low-middle and middle-high; (3) Based on the matched feature points, the transformation matrix between the images is computed by using affine transformation, and the three images, low, middle, and high, are transferred to a unified coordinate system; (4) For the superimposed edges of the images, average blending is used, but since the shooting positions of the three images are precisely designed, the edges need very little blending.

2.3. Greenhouse simulation

An accurate digital twin environment is essential for training and executing tomato harvesting decision algorithms. Therefore, we utilized the Unity engine, integrating the collected RGB and depth data to create a precise representation of the greenhouse. And we simulated the real-time growth status of tomatoes using historical data from real greenhouses, along with condition data (temperature and humidity) collected through sensors. In addition, during each harvesting session, the data collection equipment on the robot updates and calibrates the greenhouse data to ensure the accuracy of the digital twin.

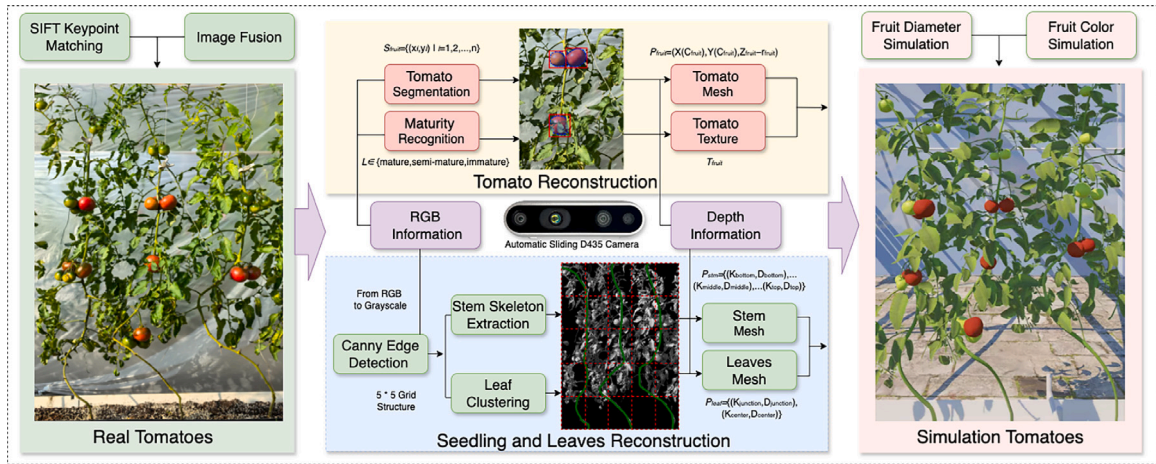


Fig. 3. The conversion process of tomato plants from real greenhouse data to virtual simulation consists of three steps, from left to right: real image capture (fused from three images), tomato and plant reconstruction, and simulation engine rendering. The tomato reconstruction is performed by segmenting and recognizing the tomato fruits to obtain spatial and ripeness information, and then reconstructing its 3D mesh using texture mapping; the plant reconstruction uses Canny edge detection and skeleton extraction to separate the main stem and reconstruct it with depth information. Finally, we reconstruct the leaf clusters based on the density of the gridded area to create a complete virtual tomato plant.

2.3.1. Data preprocessing

To convert data from the real greenhouse to the virtual greenhouse, the data preprocessing process, as shown in Fig. 3, was required to accurately reconstruct the environmental information.

In some greenhouse, background row information is difficult to remove effectively due to high leaf density or close plant spacing. In order to get more accurate spatial information, plastic sheeting can be hung at the access points of adjacent rows during the first scanning of the data in the greenhouse to ensure the accuracy of the depth information of the plants in the current row. During subsequent harvests, the plastic sheeting is no longer a mandatory selection item as the plants have stabilized and are primarily renewed with tomato fruit.

For the tomato section, the preprocessing of RGB data was achieved by the Mask R-CNN (He et al., 2017) algorithm to identify the maturity level of tomato fruits: $L \in \{\text{mature, semi-mature, immature}\}$. Simultaneously, the algorithm segmented the tomato fruit as follows:

$$S_{\text{fruit}} = \{(x_i, y_i) \mid i = 1, 2, \dots, n\},$$

The geometric center C_{fruit} and average radius r_{fruit} of the segmented tomato fruit in the image plane was computed as:

$$C_{\text{fruit}} = \left(\frac{1}{n} \sum_{i=1}^n x_i, \frac{1}{n} \sum_{i=1}^n y_i \right),$$

$$r_{\text{fruit}} = \frac{1}{n} \sum_{i=1}^n \sqrt{(x_i - C_{\text{fruit}_x})^2 + (y_i - C_{\text{fruit}_y})^2},$$

where r_{fruit} represents the mean Euclidean distance from each boundary pixel to the fruit center, reflecting the overall size of the fruit region.

By using the depth value at the geometric center, Z_{fruit} , and adding the average radius, we estimated the 3D spatial location of the tomato as:

$$P_{\text{fruit}} = (X(C_{\text{fruit}}), Y(C_{\text{fruit}}), Z_{\text{fruit}} + r_{\text{fruit}}).$$

Finally, for the texture of the tomato, we calculated the average RGB value within the segmented tomato region as T_{fruit} . Thus, the tomato's 3D mesh model with its texture was represented as $M_{\text{fruit}} = (P_{\text{fruit}}, r_{\text{fruit}}, T_{\text{fruit}})$.

For the seedling part, we first converted the image from RGB to Grey style, and then used Canny edge detection to obtain the main body of the plant, as shown in the middle part of the figure. Then, we used the skeleton extraction algorithm in OPENCV (Bradski, 2023) to separate the main stem, as shown in dark green in Fig. 3 (lower-middle). We

established the mesh projection of the plant stem based on the dark green projection, and also selected five coordinates (K_{top} , K_{ml} , K_{mid} , K_{mb} , K_{bot}) of the stem (the top, middle-top, middle, middle-bottom bottom) to obtain its depth information, and reconstructed the mesh projection in 3D space. The stem between two keypoints was smoothed with mean curvature. In this way, the information of the five key points of the stem approximated the spatial information of the stem in a real greenhouse.

For the mesh reconstruction of the leaves, we tried various ways, such as directly using the model to recognize the leaves. However, the experiments showed that it was difficult to accurately localize the number and spatial coordinates of the leaves because some of them were too small, or had a curled shape, or shaded each other. Therefore, we used leaf cluster density estimation to reconstruct the leaves. Specifically, we divided the extraction result map of the intermediate process into 5 equal parts horizontally and vertically, and estimated and reconstructed the leaf density of the region in the real scene by the density of leaf clusters in these 25 grids. We categorized the density into four classes, no leaf, sparse, normal and dense, which correspond to different leaf mesh densities. The leaf mesh of each region was connected to the main stem mesh by branches. Since the camera used three tomato plants as the minimum unit for each set of reconstructions, the leaf meshes of the regions on both sides were connected to the main stems on both sides, and the leaf meshes of the middle region were divided equally between the two main stems. Finally, the depth information D and coordinates K , of the region center/junction point with the main stem, were used to represent the leaf cluster mesh.

The color differences between tomato stems and leaves were small, so we directly used the average RGB values obtained from the sampling calculations to represent their textures. Finally, the tomato stem and leaves were represented as:

$$P_{\text{stem}} = \{(K_{\text{bot}}, D_{\text{bot}}), \dots, (K_{\text{mid}}, D_{\text{mid}}), \dots, (K_{\text{top}}, D_{\text{top}})\}$$

$$P_{\text{leaf}} = \{(K_{\text{junction}}, D_{\text{junction}}), (K_{\text{center}}, D_{\text{center}})\}$$

2.3.2. Simulation of tomato growth in unity

We selected Unity (Technologies, 2024) as the simulation engine to build a high-fidelity virtual greenhouse. The infrastructure — including the greenhouse roof, planting frames, basic lighting, ventilation, and irrigation systems — was reconstructed in Unity based on real-world measurements and structural parameters. Using the mesh-based 3D models of tomatoes (M_{fruit}) and seedlings ($M_{\text{seedlings}}$) obtained

from previous scanning steps, we instantiated numerous fruits and seedlings at their corresponding spatial coordinates, as illustrated in Fig. 3 (right).

To accurately simulate tomato growth in this virtual environment, we constructed a dynamic digital twin greenhouse model driven by real-world data. Over a six-week continuous monitoring period in a physical greenhouse, we tracked tomato growth through three major stages: immature (green), semi-mature, and fully mature (red). Environmental conditions, including temperature and humidity, were recorded via embedded sensors. Approximately 120 tomato fruits were sampled every two days using a hybrid approach combining manual measurement and automated image acquisition. Fruit diameters were precisely recorded using calipers, while over 1000 high-resolution RGB images were collected to quantify color changes. Image processing algorithms extracted average surface RGB values at each time point, enabling quantitative statistics and analysis.

Our focus was on modeling the growth process of tomato fruits after the plant had reached a relatively stable height, specifically from the point when small green tomatoes first became detectable to their development into large, mature red fruits. During this period, stem elongation and leaf senescence were relatively minimal, so our data modeling primarily focused on fruit-level changes. Moreover, during subsequent harvesting processes, the dynamic scanning system continuously updates the plant data, ensuring that any significant structural changes in the plant are promptly reflected in the digital model. Although logistic models are commonly used for simulating crop growth trends (Fang et al., 2022; Kun et al., 2023), tomato fruit development does not follow a symmetric sigmoid curve. Instead, the growth process is characteristically asymmetric. Consequently, we adopted the Gompertz function (Winsor, 1932), an asymmetric extension of the logistic model—to fit both fruit diameter and surface color trajectories.

The Gompertz function is defined as:

$$f(t) = A \cdot \exp(-b \cdot e^{-d(t-t_0)}),$$

where $f(t)$ represents either the fruit diameter or the average RGB value at time t . A denotes the upper growth limit. The parameter b serves as a scale factor, influencing the vertical displacement of the curve, while d controls the growth rate's steepness, affecting how quickly the fruit reaches its mature size or color. These parameters, in combination with the inflection point t_0 , provide a robust fit for both fruit diameter and surface color trajectories.

This fitted Gompertz model was integrated into the Unity engine through scripting interfaces, which dynamically adjusted the mesh radius and RGB color values of each tomato model over time, thus realistically simulating size and color transitions during fruit development. Although our greenhouse is equipped with real-time temperature and humidity sensors, due to the constant environmental settings used during the experiment, we did not conduct additional simulations under variable conditions. Nevertheless, the sensor infrastructure provides strong potential for extending the digital twin framework to support real-time adaptive control and cross-condition growth modeling.

2.4. Reinforcement learning

Fig. 4 illustrates the application of reinforcement learning to the tomato picking process. On the left, the training environment (virtual greenhouse) includes factors like tomato locations, ripeness, shading, and parking spots. In the middle, the reinforcement learning algorithm updates the network parameters based on rewards calculated from actions taken in the environment. On the right, after training, the model uses the learned policy to make control decisions, such as parking location, robot arm path, and whether to pick the tomato.

2.4.1. Problem formulation

The tomato harvesting task in the virtual greenhouse is modeled as a **Markov Decision Process (MDP)** represented by the tuple $\langle S, A, P, R, \gamma \rangle$, where:

• **State space S :** The state includes the following:

- Positions (x_i, y_i, z_i) of all tomatoes.
- Size σ_i and ripeness level L_i of each tomato i , where $L_i \in \{\text{mature, semi-mature, immature}\}$.
- Obstacle positions, including the coordinates of plant stems or dense foliage.
- Current parking position of the robot.

• **Action space A :** The robot performs continuous actions:

$$a_t = [\Delta x, \Delta y, \Delta z, p_t, p_{\text{position}}], \quad p_t \in [0, 1]$$

where p_t indicates a picking operation (1 for picking, 0 for no action) and p_{position} specifies the decision to adjust the parking position for optimal harvesting.

- **State transition function $P(s_{t+1}|s_t, a_t)$:** Tomatoes ripen over time, and picked tomatoes are removed. The robot's state transitions also account for any movement to new parking positions.
- **Reward function $R(s_t, a_t)$:** Encourages efficient harvesting, penalizes poor decisions, and considers the effectiveness of the chosen parking position.
- **Discount factor γ :** Balances immediate and future rewards, with $\gamma \in [0, 1)$.

The goal is to learn a policy $\pi_\theta(s)$ that maximizes the expected cumulative reward:

$$J(\theta) = \mathbb{E} \left[\sum_{t=0}^T \gamma^t R(s_t, a_t) \right]$$

2.4.2. Reward function design

The reward function $R(s_t, a_t)$ is formulated as follows:

$$R(s_t, a_t) = w_1 \cdot R_{\text{time}} + w_2 \cdot R_{\text{distance}} + w_3 \cdot R_{\text{success}} + w_4 \cdot R_{\text{ripeness}} + w_5 \cdot R_{\text{collision}} + w_6 \cdot R_{\text{position}} - w_7 \cdot R_{\text{stopping}}$$

In this formulation, each component has its own significance. The Harvest Time Reward R_{time} is defined as:

$$R_{\text{time}} = \frac{1}{\text{current harvesting time}}$$

while the Movement Distance Reward R_{distance} is given by:

$$R_{\text{distance}} = \frac{1}{\text{current movement distance}}$$

The Harvest Success Rate Reward R_{success} is structured as follows:

$$R_{\text{success}} = \begin{cases} +1, & \text{if the harvesting is successful} \\ 0, & \text{otherwise} \end{cases}$$

For the Ripeness Reward R_{ripeness} , we have:

$$R_{\text{ripeness}} = \begin{cases} +1, & \text{if the tomato is ripe} \\ 0, & \text{if the tomato is semi-mature} \\ -1, & \text{if the tomato is immature} \end{cases}$$

The Collision Penalty $R_{\text{collision}}$ is redefined to reflect the proximity between the robotic arm's end-effector and the surrounding environment of the target tomato. The modified collision penalty function is given by:

$$R_{\text{collision}} = -\log \left(1 + \frac{r_{\text{arm}}}{d_{\text{nearest}}} \right)$$

where r_{arm} denotes the radius of the robotic arm's end-effector and d_{nearest} represents the projection distance from the target tomato to the

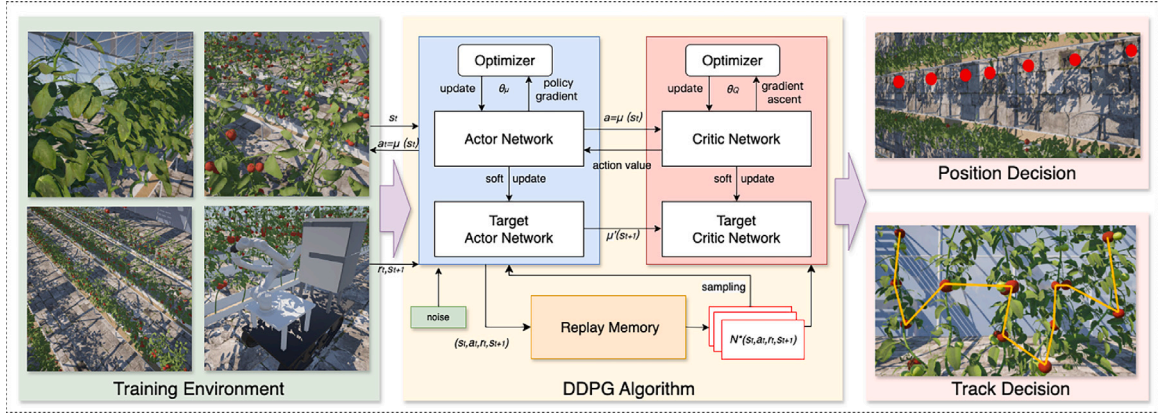


Fig. 4. The application process of reinforcement learning in tomato harvesting decision-making is illustrated. On the left is the virtual simulated greenhouse, which serves as the data foundation for reinforcement learning. This training environment enables real-time evaluation of numerous decisions made by the model. The center depicts the reinforcement learning model, adopting DDPG (Lillicrap, 2015) architecture, which is trained to optimize decision-making by adjusting parameters. On the right is the practical application, where the model determines the optimal stopping positions and local harvesting paths for the robot.

nearest stem or branch of the tomato plant. This function quantifies the risk of collision, with a smaller $d_{nearest}$ indicating a higher collision risk and a more negative $R_{collision}$.

The Parking Position Reward $R_{position}$ is defined as:

$$R_{position} = \frac{\text{ripe tomatoes} - \text{immature tomatoes}}{\text{tomatoes within the reach}}$$

This value will range from $[-1, 1]$, with higher scores for more ripe tomatoes and lower scores for more immature tomatoes. The Stopping Penalty $R_{stopping}$ is expressed as:

$$R_{stopping} = \begin{cases} 0, & \text{if no re-movement occurs} \\ -1, & \text{if a re-movement occurs} \end{cases}$$

The weights $w_1, w_2, \dots, w_7$ in the reward function are set as follows: $w_1 = 2, w_2 = 5, w_3 = 0.5, w_4 = 1, w_5 = 0.5, w_6 = 0.5, w_7 = 0.1$. These weights are chosen to balance harvesting efficiency, safety, and decision quality. In addition to this, different rewards and penalties correspond to different numerical values, which are considered along with the units when setting the weights.

2.4.3. Training procedure

The training of the tomato harvesting robot employs the Deep Deterministic Policy Gradient (DDPG) algorithm (Lillicrap, 2015). The actor and critic networks used for training are designed similarly, each consisting of an input layer, four hidden layers, and an output layer. The input layer of the actor network corresponds to the defined state space dimensions, while the output layer provides the action values. In the case of the critic network, the input consists of the combination of the current state and the chosen action, with the output being a Q-value that represents the expected return for the given state-action pair. In both networks, the number of neurons in the four hidden layers starts at 512 and progressively decreases. The activation function used for all hidden layers is ReLU. The training process employs the Adam optimizer, with an initial learning rate set to 10^{-3} and a batch size of 64.

We use robots of the same size and script driven in the simulation training environment for picking operations. For each action instruction given by the model network, the robot in the simulation environment executes it to complete the state transition. And after the transition, the corresponding reward function is calculated based on the time used, the distance moved by the robot arm, the movement of the chassis and the picking of fruit. Over M episodes, the robot interacts with the environment for T time steps, selecting actions a_t based on the current state s_t while incorporating exploration noise N_t to facilitate exploration. After each action execution, the reward r_t and the next state s_{t+1} obtained by the robot are stored in the replay buffer R .

Algorithm 1: DDPG Training Process

Input: Number of episodes M , number of steps per episode T
Output: Trained Actor and Critic networks
Initialize: Randomly initialize Critic network $Q(s, a|\theta^Q)$ and Actor network $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ ;
Initialize target networks Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$;
Initialize replay buffer R ;
for episode $i = 1$ to M **do**
 Initialize the environment and get the initial state s_1 ;
 for $t = 1$ to T **do**
 Select action $a_t = \mu(s_t|\theta^\mu) + N_t$;
 Execute action a_t , receive reward r_t and next state s_{t+1} ;
 Store transition (s_t, a_t, r_t, s_{t+1}) in R ;
 Sample a mini-batch of N transitions (s_i, a_i, r_i, s_{i+1}) from R ;
 Compute the target:

$$y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$$

 Update the Critic by minimizing the loss:

$$L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$$

 Update the Actor using the policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_{\theta^\mu} \mu(s_i|\theta^\mu) \nabla_a Q(s_i, a|\theta^Q) \Big|_{a=\mu(s_i)}$$

 Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

During each update, a mini-batch sampled from the buffer is used to update the critic and actor networks, with the objective of maximizing the cumulative reward defined by the reward function. As shown in Fig. 4 (right), through this training process, it is anticipated that significant performance improvements will be achieved, specifically manifested as a notable reduction in average harvesting time and mechanical arm movement distances, along with an increased success rate in harvesting ripe tomatoes. Additionally, the robot is expected to effectively choose optimal parking positions, enhancing the number of ripe tomatoes harvested per stop, while minimizing collisions with stems

and unnecessary re-movements, thereby improving overall harvesting efficiency.

2.5. Adaptive-exploration module

Reinforcement learning algorithms based on the Deep Deterministic Policy Gradient (DDPG) framework have demonstrated strong performance in path planning and decision-making across various domains. However, their application in agricultural scenarios requires adaptation to the unique challenges of these environments, such as high crop planting density and the significant variability in crop growth patterns. In standardized greenhouses or plant factories, crops undergo precise agronomic management, making recognition and reconstruction more reliable. Conversely, in outdoor or non-standardized cultivation settings, traditional DDPG methods may struggle to make accurate decisions due to the inherent uncertainty and variability in the data sources.

To address this challenge, we propose an Adaptive-Exploration DDPG module tailored for agricultural harvesting scenarios. This module incorporates a target recognition confidence mechanism, enabling the harvesting robot to dynamically adjust its decision-making process during reinforcement learning execution. When encountering regions with low-confidence target recognition, the robot probabilistically abandons its original strategy and leverages its onboard RGB-D camera for localized adaptive exploration (i.e., reattempting tomato detection).

Upon completing the localized recognition process, the Adaptive-Exploration module evaluates the confidence scores of the newly detected targets against those in the original decision set. If the updated recognition yields higher confidence, the system re-invokes the DDPG algorithm to optimize the harvesting sequence based on the new information. Conversely, if the original decision set remains more reliable, the robot seamlessly reverts to its prior decision path and continues execution. The detailed algorithm design is as follows:

Confidence Calculation. The harvesting robot detects tomato targets during greenhouse scene scanning and calculates the confidence score C_i as a reliability measure. The confidence score is obtained from the object detection network YOLO v8 (Ultralytics, 2024), which assigns a detection score to each identified target. Specifically, the confidence score C_i is computed as:

$$C_i = S_i \times P_i$$

where S_i represents the objectness score, indicating the probability that the detected bounding box contains an object, and P_i is the class confidence, representing the probability that the detected object belongs to the tomato category. This formulation ensures that both object existence probability and classification confidence contribute to the final confidence score, allowing the robot to assess the reliability of each detected target.

Dynamic Jump-out Mechanism. At each stopping position, the robot calculates the jump-out probability based on the tomato targets within its field of view. If triggered with probability P_{jump} , the robot abandons the original DDPG-planned path and reuses its onboard RGB-D camera for localized target detection. The probability P_{jump} is computed based on the confidence scores of all targets within the robot's current field of view:

$$P_{jump} = \alpha \cdot \frac{1}{n} \sum_{i=1}^n (1 - C_i)$$

where each C_i represents the confidence score of an individual detected target, and $\alpha = 1.2$ is a tuning parameter that adjusts the sensitivity of the jump-out mechanism.

Adaptive Exploration. When the jump-out strategy is triggered, the robotic arm captures real-time target information using the camera. A new set of confidence scores, $T' = [C'_1, C'_2, \dots, C'_n]$, is computed from the newly detected targets. This updated confidence set T' is compared

with the original decision confidence set $T = [C_1, C_2, \dots, C_n]$. The average confidence scores of both sets are calculated as $\frac{1}{n} \sum_{i=1}^n C_i$ and $\frac{1}{n} \sum_{i=1}^n C'_i$, respectively. If the new detection yields a higher average confidence score, the DDPG algorithm is re-invoked to update the harvesting sequence based on the new data.

3. Results and discussion

In the experimental session, we verified the algorithm in terms of 3D simulation effect and picking decision effect. The 3D simulation effect mainly examined the detection accuracy of tomato fruits and the reconstruction error between the 3D virtual greenhouse and the real scene. For the picking decision-making effect, we verified the algorithm performance from four parts: simulation experiment, reconstruction perturbation experiment, field experiment, and global planning experiment.

3.1. 3D reconstruction experiments

This experiment aims to evaluate the effectiveness of the proposed 3D greenhouse simulation reconstruction system in recognizing tomatoes, seedlings, and leaves, including a comparison of recognition accuracy and an assessment of reconstruction errors relative to real measurements.

3.1.1. Tomato detection

Tomato detection is fundamental to the tomato harvesting task, and accurately locating tomatoes and determining their ripeness is critical for reconstruction and harvesting operations. In this study, we randomly selected a row of tomatoes in the greenhouse for the detection experiment. By manual counting, there were 125 tomatoes in the row, including 90 ripe tomatoes, 15 half-ripe tomatoes, and 20 unripe tomatoes. The evaluation metric for detection was Mean Average Precision (mAP) and the IoU threshold was set at 0.5.

The detected data were collected by the end camera (YOLO v8 (Ultralytics, 2024)) and the slide camera (Mask R-CNN He et al., 2017), respectively. In addition to the methods used in this study, four other tomato harvesting methods (Rapado-Rincon et al., 2024; Dai et al., 2024; Jun et al., 2021; Kounalakis et al., 2021) were selected for comparison, and the experimental results of all the comparison methods were taken from their related papers, and it was ensured that all the methods used the same tomato varieties and the experimental harvests were of the same order of magnitude.

Table 1 shows the results of different tomato detection algorithms. The method in our study achieved a mAP of 91.2%, which is at the optimal level, mainly due to the design of the sliding camera, which allows tomatoes of different heights to obtain data from the best detection viewpoint. As long as a certain tomato is detected in one of the three photos taken by the slide camera, that tomato is recorded, so the recall of our method is higher than that of the traditional method. YOLO v8 achieved 88.6% mAP, which is weaker than Mask R-CNN, but its efficiency advantage is obvious due to its assembly at the end of the pick for robotic arm fine-tuning. Jun et al. (2021) designed a picking robot based on the YOLO architecture, combining it with 3D perception to achieve a mAP of 90.8%. While this result surpasses the performance of our YOLO-based method, it also comes with a longer inference time. Moreover, in terms of recognition accuracy alone, it is weaker than our Mask R-CNN approach. Rapado-Rincon et al. (2024) achieved a mAP of 85.88%, however, their greatest strength lies in the use of simulated data to support training, which significantly enhances their effectiveness in recognizing tomatoes in heavily occluded scenes. Dai et al. (2024) obtains a 82.4% mAP, but due to its multi perspective recognition design, its picking efficiency and stability are at the forefront of the field. Kounalakis's (Kounalakis et al., 2021) method is relatively old, and the constant iteration of generic recognition models makes its results no longer advantageous.

Table 1

Tomato detection accuracy comparison.

Method	mAP (%)
Ours (Mask R-CNN)	91.2
Ours (YOLO v8)	88.6
Mot-detr (Rapado-Rincon et al., 2024)	85.9
3MSP (Dai et al., 2024)	82.4
3DPME (Jun et al., 2021)	90.8
PRA (Kounalakis et al., 2021)	77.0

Table 2

Reconstruction error assessment.

Items	Average error	Maximum error
Tomato position	2.8 cm	7.5 cm
Tomato size	2.3 cm	4.1 cm
Stem position	3.1 cm	4.2 cm
Stem height	5 cm	12 cm
Tomato ripeness	3.0%	1.5%
Leaf density	7.7 pieces	13.8 pieces

Overall, the tomato detection methods used in this study are in the top tier in accuracy and provide important data support for 3D simulation reconstruction. However, due to the specificity of the fruit harvesting experiment, we cannot guarantee that the verification data of all compared methods will be exactly the same, so this experiment focuses on verifying the level of this tomato detection algorithm in the domain.

3.1.2. Reconstruction error assessment

Accurate 3D reconstruction is the basis for the training and execution of tomato picking decision-making algorithms. In order to verify the accuracy of the 3D reconstruction algorithms, we randomly selected 10 complete tomato seedlings and their fruits by manual calibration and counted the errors between the simulated simulation and the real greenhouse. We selected six items including: tomato position, size, ripeness, stem position, height, and leaf density for statistics. The average and maximum errors were analyzed. In this case, the measurement degree of accuracy for the tomato fruit is 0.5 cm and 1 cm for the tomato plant.

The experimental results, presented in Table 2, show an average tomato position error of 2.8 cm and a maximum error of 7.5 cm. It is noted that the larger errors are primarily caused by leaf shading, which leads to inaccuracies in depth measurement. However, in most cases, the shading is not severe, and the average error remains within an acceptable range. The average fruit diameter error of tomato is 2.3 cm, the maximum error is 4.1 cm. From observation, we can find that the larger error of the fruit diameter is mainly due to the overlap of multiple tomatoes can sometimes be mistakenly recognized as a large tomato. To evaluate fruit ripeness recognition, we define the average recognition error as the percentage of all recognition errors (e.g., unripe and half-ripe classified as fully ripe) and the maximum error as the percentage of unripe classified as fully ripe. Experimental results show that unripe and half-ripe tomatoes combined were mistakenly recognized as ripe in only 3.0% of cases, with unripe tomatoes accounting for just 1.5%. This demonstrates that the ripeness recognition error is minimal.

For stem position, we measure the spatial error of the bottom and middle keypoints. The average error is 3.1 cm, and the maximum error is 4.2 cm. This indicates that the variation in stem identification is small. The error mainly results from some stems being too thin or the roots being twisted, which affects identification accuracy. The height of the stem is recognized by the distance from the top keypoint to the bottom keypoint, the average error is 5 cm, the maximum error is 12 cm. This is mainly due to the top of the stem is generally very thin or with leaf clusters, so it is difficult to capture a specific point. The leaf density evaluation shows an average error of 7.7 leaves and a maximum error of 13.8 leaves, for one complete tomato plant. The larger errors

are primarily caused by smaller or partially unfolded leaves, which lead to fewer green pixels being extracted during segmentation, resulting in a mis-classification of lower leaf density. However, these small leaves have minimal impact on the robotic arm's movement during actual picking operations. Moreover, considering 7.7 leaves dispersed across more than ten leaf cluster grids, the error in each region is actually less than one leaf.

In summary, the 3D simulation reproduced the real greenhouse scene very well, especially the location of tomato and stems. It is also clear from this experiment that the reconstruction simulation algorithm has the potential to be applied to other greenhouse crops or spherical-like outdoor crop picking decisions in the future as well.

3.2. Harvesting experiments

To evaluate the effectiveness of the proposed decision-making algorithm for tomato harvesting, we designed a four-part experiment. First, a comparative study was conducted using the greenhouse simulation platform where we can evaluate the performance of a virtual harvesting robot before and after applying the decision algorithm. In the second part, we conducted a real-life tomato harvesting experiment at a greenhouse in Beijing, China, and we also compared the efficiency of our system with the published results of other tomato picking robots in the field. Finally, we also demonstrated global decision-making capabilities through two cases.

Evaluation metrics include:

- **Harvested tomatoes:** Total number of successfully harvested ripe tomatoes.
- **Remained tomatoes:** Number of tomatoes left unharvested.
- **Branch collisions:** Total number of collisions with branches during harvesting.
- **Average time:** Average time per harvesting operation.
- **Average arm movement:** Average distance traveled by the robotic arm during harvesting.

3.2.1. Simulation experiments

In the simulation experiment, we created a tomato-picking robot in the Unity engine with the same proportional size, as shown in Fig. 5 left. We used the same tracked chassis control script, robotic arm control script, and tomato recognition & segmentation script to control the simulated greenhouse harvesting robot. This approach aimed to replicate the real-world tomato-picking process as accurately as possible. The right figure shows the picking operation of the robotic arm, whose picking is performed by the rotary processor at the end, with an average of 10 clockwise rotations, approximately takes 0.8 s. In addition, the speed of the robot and its arm movements was also consistent with real devices. It is common for the robotic arm to collide with seedlings during field harvesting, and these collisions typically require additional time to reposition the arm to a safe location. We counted the results of 100 real-life collisions with seedlings and calculated the average return time to be about 2.9 s, which takes into account situations such as entangled seedlings. Therefore, when a collision occurs during the picking experiment, we performed a 2.9-s robotic arm reset action to return the robotic arm end-effector to its initial position.

The experiment was conducted in two groups, with each group executing a full task defined as completing the harvest of a single row. Since in the simulation environment the harvested tomatoes can be reset, both groups randomly selected the same row in the simulated greenhouse, as shown in Fig. 6. During the experiment, we recorded the number of tomatoes harvested, the time taken, the distance traveled, the proportion of untouched tomatoes, and the number of collisions by the robotic arm.

The decision-free algorithm robot used in the experiments has a picking logic of sequential scanning (via a depth camera on the robot arm). As an example, it will sequentially recognize and pick tomatoes



Fig. 5. The rendered image of the reconstructed virtual greenhouse in the Unity engine. The tomato picking robot on the left is running with the same size as the real hardware device. On the right side, a close-up view of the picking process is displayed, which uses the same decision control algorithm as the real robot. In addition, the simulation includes modeling the growth process of tomatoes based on accumulated greenhouse data, where the size and color of tomatoes gradually change over time.

Table 3

Comparison between basic DDPG and Adaptive-Exploration DDPG under different reconstruction perturbation errors in terms of average harvest time (s).

Perturbation level (cm)	Basic DDPG	Adaptive-Exploration DDPG
2.5	7.6	8.1
5.0	8.0	8.3
7.5	8.6	8.7
10.0	9.5	9.3

from left to right along the x -axis direction. Note that this depth camera, like the vast majority of picking robot designs in the field, has a relatively limited field of view because it does not have a slider. It is therefore necessary to scan in front of this parking position by moving the robot arm to see if there are any tomatoes left to pick from top to bottom. When all the ripe tomatoes have been picked, continue to the right.

When comparing the picking visualization results, our method demonstrates clear advantages over the algorithm without picking decisions. It produces shorter total lengths of orange connecting lines, while the connecting lines of the latter are noticeably more curved or zigzag. During the picking process, our method skipped six tomatoes located on the inside, whereas the comparison algorithm skipped almost none. For instance, in the 4–6 m and 12–14 m intervals, our method avoided picking inner tomatoes, reducing collisions and keeping average picking time within 8 s, as shown in line 3. In contrast, the comparison method, penalized by collision-induced delays, exceeds 12 s on average. However, the comparison algorithm only harvested two of these skipped tomatoes, as indicated in line 6. Moreover, as seen in line 5, picking these two tomatoes resulted in three additional collisions compared to our method. The 4–8 m segments provide further evidence of the decision algorithm's effectiveness in reducing the robotic arm's travel distance by optimizing stopping positions and picking sequences. As shown in line 4, the average travel distances for our method in these two segments are 31 and 35 cm, while the comparison method requires 42 and 41 cm.

Overall, our method consistently outperforms in key metrics such as average picking time (line 3) and average robotic arm travel distance (line 4), maintaining an advantage across almost all intervals. In terms of averages, our method achieves a picking time of approximately 7 s compared to 11 s for the comparison method. Similarly, the average travel distance is 33 cm for our method, versus 41 cm for the comparison method. Additionally, our method achieves smoother performance, as evidenced by the significantly smaller variance in picking times. The comparison method relies solely on sequential scanning, resulting in more collisions and inefficient attempts to pick inside tomatoes in the absence of reconstructed spatial perception or decision support. In addition, the limited viewing angle of the robot arm camera limits the speed of its tomato location perception from the hardware side. The simulation data therefore clearly shows that our approach achieves a better balance between efficiency, safety, and yield.

3.2.2. Reconstruction perturbation experiment

To validate the effectiveness of the Adaptive-Exploration module of our DDPG algorithm, we conducted a reconstruction perturbation experiment within a simulated environment. Perturbations were introduced to three key scanned crop model parameters: fruit size, fruit position, and stem position. These perturbations were categorized into four levels—2.5 cm, 5.0 cm, 7.5 cm, and 10.0 cm. The evaluation focused on a single metric, Average Harvest Time (s), to assess the performance of the two algorithms under different levels of perturbation.

As shown in Table 3, under a perturbation error of 2.5 cm, the basic DDPG algorithm demonstrates slightly better performance in terms of Average Harvest Time compared to the Adaptive-Exploration DDPG. With low perturbation, the reconstructed crop data is highly accurate, and the initial planned path remains nearly optimal, requiring minimal adjustments during execution. In this case, the additional exploration overhead of the Adaptive-Exploration DDPG does not yield substantial improvements. At the 5.0 cm perturbation error, the performance gap between the two algorithms begins to narrow. The basic DDPG still maintains a slight advantage, but incurs additional time compensating for target deviations due to reconstruction errors. The Adaptive-Exploration DDPG, on the other hand, faces greater time costs due to frequent re-scanning and path reassessments. Under a 7.5 cm perturbation error, the performance difference continues to shrink, and at 10.0 cm perturbation error, the Adaptive-Exploration DDPG outperforms the basic DDPG. At this higher error level, the basic DDPG struggles with repeated corrections due to path deviations from reconstruction inaccuracies. In contrast, the Adaptive-Exploration DDPG dynamically refines the trajectory, efficiently adapting to the uncertain environment and reducing unnecessary path corrections. This comparison shows that while the basic DDPG excels in low-error conditions, its performance declines as uncertainty increases. The Adaptive-Exploration DDPG, however, becomes more advantageous in environments where reconstruction errors significantly impact path planning.

3.2.3. Field experiments

The real-world experiment followed a similar structure to the simulation. Two groups were assigned to harvest a full row of tomatoes, with one group using the decision system and the other harvesting without it. In the reconstruction error experiment, we observed that the reconstruction errors for fruit position, fruit size, and stem position were 2.3 cm, 2.8 cm, and 3.1 cm, respectively—all significantly smaller than 5 cm. Therefore, in this strictly managed, agronomically standardized greenhouse environment, the basic DDPG framework was already capable of achieving optimal decision-making performance. As a result, we did not include a comparison with the adaptive exploration module in this part of the experiment. Due to the condition of the real greenhouse, it was not feasible to harvest the same row for both groups. To ensure comparability, we manually counted the number of tomatoes, the ripeness ratio, and the number of tomatoes that could not

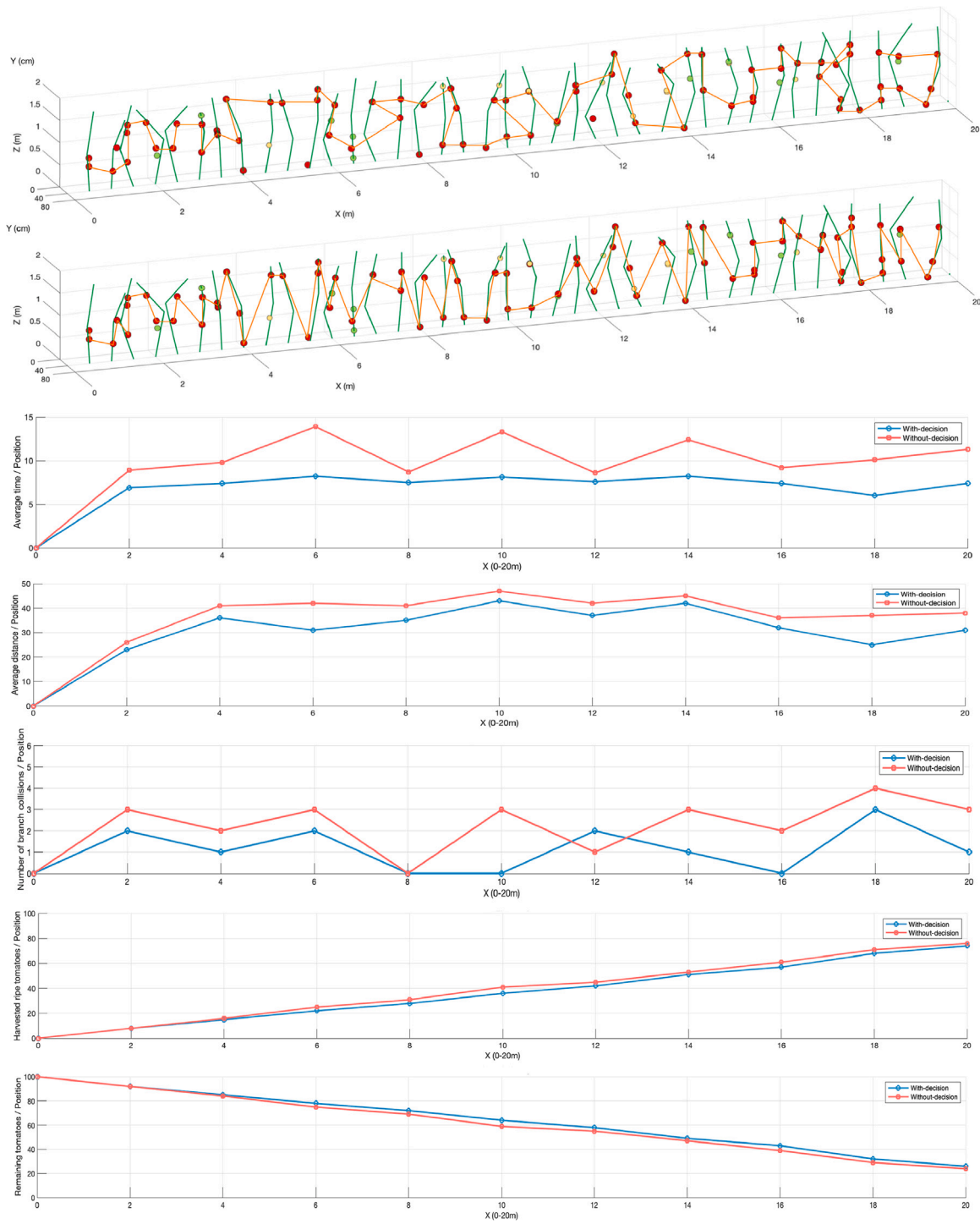


Fig. 6. The results of simulation harvesting experiment. The first and second rows show the tomato picking decision algorithm we proposed, versus the picking execution graph without decision algorithm, respectively. The green connecting lines represent tomato stems, and we have hidden the leaves for ease of presentation. The red, yellow, and green colors represent ripe, semi-ripe, and unripe tomatoes. The orange connecting lines represent the picking paths. The third to seventh rows are: the average picking time, the average distance traveled by the robot arm, the average number of collisions, the number of tomatoes picked, and the number of tomatoes remaining.

be automatically picked for each row before starting the experiments. The statistical differences in these three metrics across the two rows were all less than 3%, ensuring consistent experimental conditions. Fig. 7 shows a full view of the robot harvesting in the field, accompanied by a close-up of the picking robot arm in action.

As shown in Table 4, our method achieves an average picking time of 7.4 s, significantly outperforming the 11.3 s of the comparison method. This result is also close to the simulated environment's values of 7.1 and 11.0 s. The small difference of approximately 0.3 s in the

simulated environment may be attributed to variations in the firmness of the connection between some tomatoes and seedlings, affecting the picking action's average duration. But the picking time advantage is still evident due to the gains from better paths, better picking options, and wider lens field of view. In terms of the average distance traveled by the robotic arm, our method demonstrates a clear advantage at 35 cm compared to the comparison method. This is consistent with the 33 cm average in the simulated environment, although minor differences may arise from variations in row difficulty between the simulated



Fig. 7. Harvesting robots in field experiments. The left side shows the overall image of the robot, and the scanning/decision-making system can be visualized on the screen; On the right is a close-up of the robotic arm picking tomatoes.

Table 4
Comparison of harvest metrics in field experiments.

Metric	Ours	Baseline
Harvest count (counts)	114	119
Average harvest Time (s)	7.4	11.3
Average harvest distance (cm)	34	43
Collision count (times)	17	31
Missed harvest count (counts)	15	12

Table 5
Harvest time and Success rate for different methods.

Method	Harvest time (s)	Success rate (%)
Ours	7.4	85.3
Novel-EE	15.5	80.6
3MSP2	8.6	70.9
Pose-Rec	14.6	72.1
PID	15.0	86.0
HIS	24.0	83.9

and real environments. Despite these differences, the efficiency of our method remains evident. Regarding the number of ripe tomatoes picked and the number of tomatoes left behind, the difference between the two methods is within 5%, which aligns closely with results from the simulated environment. Both our and baseline methods collected a few tomatoes (4 and 3) at the trans-ripening stage, likely due to recognition errors caused by occlusions during the identification process. For the number of collisions, the gap between our method and the comparison method is still close to twice in real-world experiments. From the experimental process, it can be seen that the occurrence of multiple collisions within a single area has increased. This increase is likely due to the more complex leaf shading in the real environment. Importantly, the more challenging the environment, the greater the value of effective decision-making.

Overall, the results of the simulation and real-world experiments are highly consistent. Although the performance of some metrics is weaker than that of the simulation environment, the gap with the comparison method is also larger, reflecting the superiority of our method in complex environments. This highlights the critical role of the reinforcement learning model and the importance of decision-making in optimizing the picking process.

3.2.4. Comparisons with other works

We also conducted a comparative experiment to evaluate our methods against those from other studies. Specifically, we compared five approaches: the methods presented in Novel-EE (Park et al., 2023), 3MSP2 (Dai et al., 2024), Pose-Rec (Rong et al., 2022), PID (Lili et al.,

2017), and HIS (Feng et al., 2015). Due to slight differences in growing environments across the studies, the comparison focused primarily on two key metrics: the time required to pick a single tomato and the tomato-picking success rate.

It is important to note that the tomato harvesting tasks were conducted in separate greenhouse scenarios. We selected similar tomato varieties (all normal size tomatoes, not cherry tomatoes), comparable planting densities (excluding sparse experimental scenarios such as mock tomato simulations), and harvesting arms of similar efficiency (excluding two-armed robots). However, even with these controls, it is impossible to ensure identical experimental difficulty across all scenarios. Therefore, the primary goal of this experiment is to evaluate the trend of improvement in harvesting efficiency using the decision algorithm.

The results in Table 5 show that our method consistently outperforms others in both harvest time and success rate, achieving an average harvest time of 7.4 s and a success rate of 85.3%. This performance is significantly better than Novel-EE (Park et al., 2023) and HIS (Feng et al., 2015), which required 15.5 and 24 s, respectively, and comparable to Lili et al. (2017), which achieved 86% success but with a much longer harvest time of 15 s. While 3MSP2 (Dai et al., 2024) and Pose-Rec (Rong et al., 2022) demonstrate shorter harvest times (8.6 and 14.6 s, respectively), their lower success rates of around 71% highlight challenges in optimizing both speed and accuracy. However, considering that 3MSP2's (Dai et al., 2024) results were obtained without collision, it also has strong practicality.

Our approach effectively balances efficiency and accuracy, thanks to the decision-making algorithm that optimizes the picking sequence and robotic arm motion, reducing unnecessary movements and collisions. It also highlights the advantage of sliding data scanning cameras in future 3D-based greenhouse harvesting. Compared to a “semi-fixed” camera on the robotic arm, our method outperforms in both timeliness and accuracy, confirming its superiority in achieving speed and reliability in tomato harvesting.

3.2.5. Global decision experiments

Using our dynamic simulation system, our method has two other unique advantages. On the one hand, it can make a decision on the start time of harvesting based on the dynamic growth state of the tomatoes and give the order of harvesting for each row in the whole greenhouse, as shown in Fig. 8 (left). In the current tomato harvesting process, it is very important to choose the right time for harvesting. If it is too early in the season, the tomatoes may not be fully ripe and many tomatoes that are recognized as unripe will be skipped. If the picking time is too late, some tomatoes may be too ripe and there may even be isolated cases of dropping and rotting, resulting

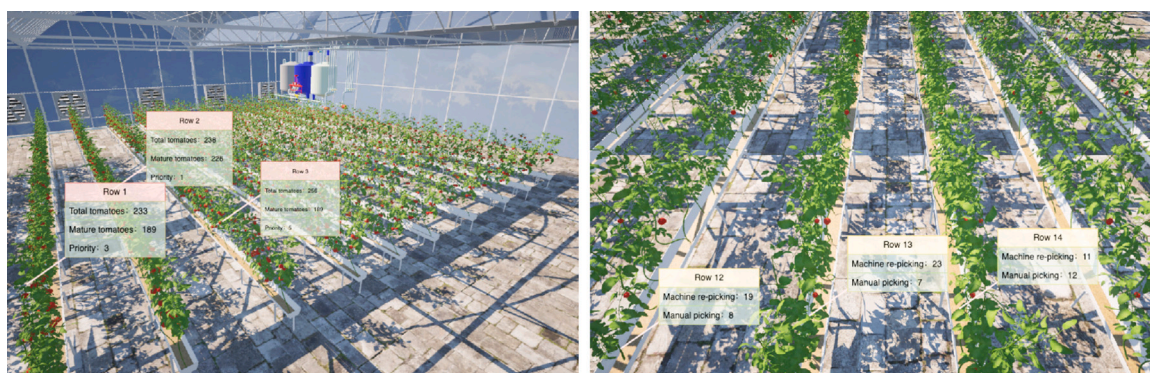


Fig. 8. Left: The picking decision results from a global perspective, with the system displaying the total number of tomatoes in each row, the number of ripenings, and the priority of picking. When the overall maturity reaches the recommended range, the system will drive the robot to start picking. On the right: For tomatoes skipped during harvesting, detailed records will be made in the system, with each line displaying the number of missed tomatoes and their specific location coordinates, making it convenient for subsequent harvesting.

in economic losses. With extensive experience of picking, it is not difficult to determine this time, but for very large greenhouses and for workers with relatively limited experience, choosing the right time to pick and the order in which to pick the different rows of tomatoes becomes a complex problem. Through our analysis of the historical ripening trend of tomatoes in greenhouses and the efficiency of robot harvesting. It can be calculated that 72% to 84% of tomato ripening in the greenhouse is the ideal starting stage for harvesting. Within this range, enough mature tomatoes can be harvested without worrying about over ripening and rotting. By using our system, the overall maturity of tomatoes in the greenhouse can be automatically monitored until they enter the recommended picking range.

On the other hand, as mentioned in the above experiments, our method can record the missed picking of ripe tomatoes as shown in Fig. 8 (right) and generate a manual picking sequence, which can efficiently compensate for the small disadvantage of the picking success rate. The number of missed tomatoes in each row and their coordinates can be visualized on the screen. The operator can either operate the robot to go into the back row to complete the picking, or he can do it manually. This would be much more convenient and safer than reaching in from the front and risking damaging the seedlings.

This experiment demonstrates the advantages of the 3D simulation-based picking decision algorithm, surpassing traditional methods in capacity limits. The 3D environment enhances spatial awareness and dynamic capabilities, improving decision-making accuracy and intelligence, and paving the way for future advancements in picking robots.

4. Conclusion

Our work proposes a digital twin-driven system for efficient tomato harvesting in greenhouses. A sliding depth camera mounted on the picking robot enables automatic scanning and identification of tomato positions, seedling distribution, fruit size, and ripeness throughout the greenhouse environment. The collected data are processed through computer vision algorithms to construct an accurate digital twin model, which is then rendered in a dynamic 3D virtual greenhouse using the Unity engine. Leveraging this high-fidelity virtual environment, we employ a reinforcement learning algorithm based on the DDPG architecture to train the robot to autonomously execute efficient harvesting paths while optimizing task execution strategies in complex greenhouse scenarios.

The proposed system addresses key challenges in greenhouse tomato harvesting. First, it solves the limited-view problem by using an automated sliding-depth camera to construct a comprehensive digital twin, enabling decision-making from a global–local multi-view perspective rather than relying on a fixed local view. Second, reinforcement learning models trained within the twin environment optimize stop

points, picking paths, and adaptive strategies, improving efficiency and reducing loss—particularly by learning to skip hard-to-reach fruits and schedule them for manual picking. Third, the digital twin simulates tomato growth dynamics globally, supporting data-driven decisions on harvesting timing and priority, moving beyond experience-based judgment.

Experimental results demonstrate that the system achieves outstanding harvesting performance in both simulated and real-world environments. The average harvesting time is maintained within 7.4 s, representing a 34.95% improvement in efficiency compared to baseline systems without learning strategies. Additionally, the robotic arm's movement distance is reduced by 20.93%, and the number of collisions decreases by 45.16%. These improvements are realized while maintaining comparable total harvest counts and picking success rates, achieving a well-balanced performance across speed, stability, and accuracy. Moreover, our digital twin greenhouse model exhibits a high level of consistency with the real environment in terms of the spatial position, geometric size, and ripeness status of tomato fruits and seedlings, providing a robust foundation for subsequent algorithm training and system optimization.

In our study, certain limitations should be acknowledged. While the adaptive exploration module helps mitigate reconstruction errors, the effectiveness of our method still depends on the standardization level of the greenhouse. Environments with low standardization or small-scale planting may not provide the necessary precision in scanning, simulation, and sensing, making the approach less suitable. Additionally, the applicability of our method varies across different fruit types. It performs well on greenhouse-grown, near-spherical fruits, while cluster fruits such as cherry tomatoes or grapes may require specialized end-effectors for effective harvesting. For outdoor orchards, where fruits are often heavily obstructed by branches, the current approach remains challenging and requires further research.

In conclusion, our work is the first to propose dynamic reconstruction of tomato greenhouses based on digital twin technique, driving the training of reinforcement learning models to optimize the picking decision process. This effectively improves picking efficiency, reduces seedling loss and fruit rot rate. Due to the generalization of the digital twin algorithm, our approach has the potential to be extended to picking decisions for other greenhouse crops in the future, creating greater value in agricultural production.

CRedit authorship contribution statement

Yining Lang: Writing – original draft, Visualization, Methodology, Formal analysis, Conceptualization. **Yanqi Zhang:** Software. **Tan Sun:** Writing – review & editing, Conceptualization. **Xiujuan Chai:** Writing – review & editing, Supervision, Project administration, Funding acquisition. **Ning Zhang:** Resources, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was funded by the Beijing Smart Agriculture Innovation Consortium Project (BAIC10-2025), the Central Public-interest Scientific Institution Basal Research Fund (Y2022QC17), Innovation Program of Chinese Academy of Agricultural Sciences (CAAS-CAE-202302, CAAS-BRC-SAE-2025-01, CAAS-ASTIP-2024-AII), the Agricultural Independent Innovation of Jiangsu Province (CX(24)1021), and the Scientific Research and Innovation Project of Northern Agricultural and Livestock Husbandry Technology Innovation Center of CAAS under grant number BFGJ2022004.

Data availability

The data that has been used is confidential.

References

- Bradski, G., 2023. OpenCV. URL: <https://opencv.org/>.
- Dai, N., Fang, J., Yuan, J., Liu, X., 2024. 3MSP2: Sequential picking planning for multi-fruit congregated tomato harvesting in multi-clusters environment based on multi-views. *Comput. Electron. Agric.* 225, 109303. <http://dx.doi.org/10.1016/j.compag.2024.109303>.
- Dong, L., Zhu, L., Zhao, B., Wang, R., Ni, J., Liu, S., Chen, K., Cui, X., Zhou, L., 2025. Semantic segmentation-based observation pose estimation method for tomato harvesting robots. *Comput. Electron. Agric.* 230, 109895.
- Fang, S.-L., Kuo, Y.-H., Kang, L., Chen, C.-C., Hsieh, C.-Y., Yao, M.-H., Kuo, B.-J., 2022. Using sigmoid growth models to simulate greenhouse tomato growth and development. *Hortic.* 8 (11), 1021.
- Feng, Q., Wang, X., Wang, G., Li, Z., 2015. Design and test of tomatoes harvesting robot. In: 2015 IEEE International Conference on Information and Automation. IEEE, pp. 949–952. <http://dx.doi.org/10.1109/ICInfA.2015.7279423>.
- Firdaus, H., Mardiaty, R., Setiawan, A.E., 2024. Tomato harvesting robot prototype: Fuzzy-controlled arm with vision-based tomato detection. In: 2024 10th International Conference on Wireless and Telematics. ICWT, IEEE, pp. 1–6.
- Gong, L., Wang, W., Wang, T., Liu, C., 2022. Robotic harvesting of the occluded fruits with a precise shape and position reconstruction approach. *J. Field Robot.* 39 (1), 69–84. <http://dx.doi.org/10.1002/rob.22041>.
- He, K., Gkioxari, G., Dollár, P., Girshick, R., 2017. Mask R-CNN. In: Proceedings of the IEEE International Conference on Computer Vision. ICCV, pp. 2961–2969. <http://dx.doi.org/10.1109/ICCV.2017.322>.
- Jun, J., Kim, J., Seol, J., Kim, J., Son, H.I., 2021. Towards an efficient tomato harvesting robot: 3D perception, manipulation, and end-effector. *IEEE Access* 9, 17631–17640. <http://dx.doi.org/10.1109/ACCESS.2021.3052240>.
- Kim, J., Pyo, H., Jang, I., Kang, J., Ju, B., Ko, K., 2022. Tomato harvesting robotic system based on Deep-ToMaToS: Deep learning network using transformation loss for 6D pose estimation of maturity classified tomatoes with side-stem. *Comput. Electron. Agric.* 201, 107300. <http://dx.doi.org/10.1016/j.compag.2022.107300>.
- Kounalakis, N., Kalykakis, E., Pettas, M., Makris, A., Kavoussanos, M.M., Sfakiotakis, M., Fasoulas, J., 2021. Development of a tomato harvesting robot: Peduncle recognition and approaching. In: HORA. IEEE, pp. 1–6. <http://dx.doi.org/10.1109/HORA52670.2021.9461281>.
- Kun, Z., Pingzeng, L., Ze, Z., Yan, Z., Feng, M., 2023. Digital research on growth process of facility tomato based on Logistic model. *J. Chin. Agric. Mech.* 44 (9), 72.
- Li, Y., Feng, Q., Zhang, Y., Peng, C., Ma, Y., Liu, C., Ru, M., Sun, J., Zhao, C., 2024b. Peduncle collision-free grasping based on deep reinforcement learning for tomato harvesting robot. *Comput. Electron. Agric.* 216, 108488.
- Li, Y., Feng, Q., Zhang, Y., Peng, C., Zhao, C., 2024c. Intermittent stop-move motion planning for dual-arm tomato harvesting robot in greenhouse based on deep reinforcement learning. *Biomimetics* 9 (2), 105.
- Li, Y., Feng, Q., Zhang, Y., Peng, C., Zhao, C., 2024d. Intermittent stop-move motion planning for dual-arm tomato harvesting robot in greenhouse based on deep reinforcement learning. *Biomimetics* <http://dx.doi.org/10.3390/biomimetics9020105>.
- Li, X., Ma, N., Han, Y., Yang, S., Zheng, S., 2024a. Ahppebot: Autonomous robot for tomato harvesting based on phenotyping and pose estimation. <http://dx.doi.org/10.48550/arXiv.2405.06959>, arXiv preprint arXiv:2405.06959.
- Lili, W., Bo, Z., Jinwei, F., Xiaon, H., Shu, W., Yashuo, L., Zhou, Q., Chongfeng, W., 2017. Development of a tomato harvesting robot used in greenhouse. *Int. J. Agric. Biol. Eng.* 10 (4), 140–149. <http://dx.doi.org/10.1109/HORA52670.2021.9461281>.
- Lillicrap, T., 2015. Continuous control with deep reinforcement learning. <http://dx.doi.org/10.48550/arXiv.1509.02971>, arXiv preprint arXiv:1509.02971.
- Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L., 2025. Microsoft coco: Common objects in context. <https://cocodataset.org/>.
- Liu, L., Yang, Q., He, W., Yang, X., Zhou, Q., Addy, M.M., et al., 2024b. Design and experiment of nighttime greenhouse tomato harvesting robot. *J. Eng. Technol. Sci.* 56 (3), 340–352.
- Liu, J., Zheng, Y., Pi, J., Zhou, C., Kong, J., 2024a. Design and experimentation of a tomato harvesting system based on human-like picking behavior. In: International Conference on Computing, Control and Industrial Engineering. Springer, pp. 301–319.
- Lowe, D.G., 1999. Object recognition from local scale-invariant features. In: Proc of IEEE International Conference on Computer Vision. p. 1. <http://dx.doi.org/10.1109/ICCV.1999.790410>.
- Luo, Y., Wang, J., Xing, J., Yang, L., Zhang, J., 2023. Design of a flexible end-effector for a tomato harvesting robot. In: 2023 IEEE International Conference on Unmanned Systems. ICUS, IEEE, pp. 389–394.
- Miao, Z., Yu, X., Li, N., Zhang, Z., He, C., Li, Z., Deng, C., Sun, T., 2023. Efficient tomato harvesting robot based on image processing and deep learning. *Precis. Agric.* 24 (1), 254–287. <http://dx.doi.org/10.1007/s11119-022-09944-w>.
- Park, Y., Seol, J., Pak, J., Jo, Y., Jun, J., Son, H.I., 2023. A novel end-effector for a fruit and vegetable harvesting robot: mechanism and field experiment. *Precis. Agric.* 24 (3), 948–970. <http://dx.doi.org/10.1007/s11119-022-09981-5>.
- Qin, Z., Fengpu, L., Xianping, J., Zheng, X., Can, X., 2021. Motion planning method and experiments of tomato bunch harvesting manipulator. *Trans. Chin. Soc. Agric. Eng.* 37 (9), 149–156. <http://dx.doi.org/10.11975/j.issn.1002-6819.2021.09.017>.
- Rapado-Rincon, D., Nap, H., Smolenova, K., van Henten, E.J., Kootstra, G., 2024. MOT-DETR: 3D single shot detection and tracking with transformers to build 3D representations for agro-food robots. *Comput. Electron. Agric.* 225, 109275. <http://dx.doi.org/10.1016/j.compag.2024.109275>.
- Rong, J., Wang, P., Wang, T., Hu, L., Yuan, T., 2022. Fruit pose recognition and directional orderly grasping strategies for tomato harvesting robots. *Comput. Electron. Agric.* 202, 107430. <http://dx.doi.org/10.1016/j.compag.2022.107430>.
- Sengaphone, P., De Leon, J.M., Concepcion, R., Bandala, A.A., Augusto, G.L., Naguib, R., Gonzaga, J.A., Chua, J.A., Gan Lim, L.A., 2024. End-effectors of the robotic arms for tomato harvesting: A comprehensive review. *J. Comput. Appl. Res. Mech. Eng. (JCARME)* 14 (1), 1–18.
- Su, L., Liu, R., Liu, K., Li, K., Liu, L., Shi, Y., 2023. Greenhouse tomato picking robot chassis. *Agric.* 13 (3), 532.
- Sun, T., Zhang, W., Miao, Z., Zhang, Z., Li, N., 2023. Object localization methodology in occluded agricultural environments through deep learning and active sensing. *Comput. Electron. Agric.* <http://dx.doi.org/10.1016/j.compag.2023.108141>.
- Technologies, U., 2024. Unity. URL: <https://unity.com/>.
- Ultralytics, 2024. Ultralytics repository. <https://github.com/ultralytics/ultralytics>. (Accessed 21 November 2024).
- Winsor, C.P., 1932. The Gompertz curve as a growth curve. *Proc. Natl. Acad. Sci.* 18 (1), 1–8.
- Yin, H., Chai, Y., Yang, S.X., Mittal, G.S., 2009. Ripe tomato recognition and localization for a tomato harvesting robotic system. In: 2009 International Conference of Soft Computing and Pattern Recognition. IEEE, pp. 557–562. <http://dx.doi.org/10.1109/SoCPar.2009.111>.
- Yonghong, Z., Yuchao, L., Tiantian, D., Xiayang, Q., Yunping, L., Jingxing, C., 2024. Target identification and detection for tomato harvesting robot in unstructured environments. *J. Chin. Agric. Mech.* 45 (4), 205.
- Zheng, X., Rong, J., Zhang, Z., Yang, Y., Li, W., Yuan, T., 2024. Fruit growing direction recognition and nesting grasping strategies for tomato harvesting robots. *J. Field Robot.* 41 (2), 300–313.